

Name: V.Siri Chandana
Roll No:2520030609
Section:2
Subject:OS

Skill Programs

8A.

```
chinn@LAPTOP-RDULKNPB: ~ × + ▾
GNU nano 8.7.1 Skill8.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a, *b;

    a = (int *)malloc(5 * sizeof(int));

    b = (int *)calloc(5, sizeof(int));

    a = (int *)realloc(a, 10 * sizeof(int));

    free(a);
    free(b);

    return 0;
}
```

Output:

```
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ nano Skill8.c
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ gcc Skill8.c -o Skill8
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ ./Skill8
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$
```

8B.

```
chinn@LAPTOP-RDULKNPB: ~ × + v
GNU nano 8.7.1 Skill8b.c *
#include <stdio.h>
#include <unistd.h>

int main()
{
    int x = 100;

    if (fork() == 0)
    {
        printf("Child Before = %d\n", x);

        x = 500;

        printf("Child After = %d\n", x);
    }
    else
    {
        sleep(2);

        printf("Parent = %d\n", x);
    }

    return 0;
}
```

Output:

```
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ nano Skill8b.c
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ gcc Skill8b.c -o Skill8b
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ ./Skill8b
Child Before = 100
Child After = 500
Parent = 100
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ |
```

9.

```
chinn@LAPTOP-RDULKNPB: ~ × + v
GNU nano 8.7.1 Skill9.c *
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main() {
    char path[PATH_MAX];
    char current[PATH_MAX];
    char previous[PATH_MAX] = "";

    while (1) {
        getcwd(current, sizeof(current));
        printf("\nCurrent Directory: %s\n", current);

        printf("Enter directory (type exit to quit): ");
        fgets(path, sizeof(path), stdin);
        path[strcspn(path, "\n")] = '\0';

        if (strcmp(path, "exit") == 0)
            break;

        if (strcmp(path, "-") == 0) {
            if (strlen(previous) == 0) {
                printf("No previous directory.\n");
                continue;
            }
            strcpy(path, previous);
        }

        strcpy(previous, current);

        if (chdir(path) == 0) {
            getcwd(current, sizeof(current));
            printf("Directory Changed Successfully.\n");
            printf("New Directory: %s\n", current);
        } else {
            perror("Error");
        }
    }

    return 0;
}
```

Output:

```
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ nano Skill9.c
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ gcc Skill9.c -o Skill9
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ ./Skill9

Current Directory: /home/chinn/OS_SKILL
Enter directory (type exit to quit): /home/hp
Error: No such file or directory

Current Directory: /home/chinn/OS_SKILL
Enter directory (type exit to quit): exit
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ |
```

10.

```
chinn@LAPTOP-RDULKNPB: ~ × + ▾
GNU nano 8.7.1 Skill10.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <limits.h>
#include <sys/wait.h>

int validName(char *name)
{
    if (!isalpha(name[0]) || name[0] == '_')
        return 0;

    for (int i = 1; name[i]; i++)
    {
        if (!isalnum(name[i]) || name[i] == '_')
            return 0;
    }
    return 1;
}

int main()
{
    char input[200];
    char cwd[PATH_MAX];
    char lastPath[PATH_MAX];

    while (1)
    {
        if (getcwd(cwd, sizeof(cwd)) != NULL)
        {
            strcpy(lastPath, cwd);
            printf("\nCurrent Directory: %s\n", cwd);
        }
        else
        {
            perror("getcwd");
            break;
        }

        printf("\nCommands:\n");
        printf("  pwd\n");
        printf("  export NAME=value\n");
        printf("  child\n");
        printf("  exit\n");

        // --- Inferred Input Reading Logic (Cropped between images) ---
        printf("Enter command: ");
        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }
        input[strcspn(input, "\n")] = '\0'; // Remove newline

        if (strcmp(input, "pwd") == 0)
        {
            printf("%s\n", cwd);
        }
        else if (strcmp(input, "exit") == 0)
        {
            printf("Exiting program...\n");
            break;
        }
        // -----
        /* Child Process */
        else if (strcmp(input, "child") == 0)
        {
            pid_t pid = fork();

            if (pid == 0)
            {
                execl("/usr/bin/env", "env", NULL);
                perror("exec");
                exit(1);
            }
        }
    }
}
```



chinn@LAPTOP-RDULKNPB: ~



GNU nano 8.7.1

Skill10.c *

```
    else
    {
        perror("getcwd");
        break;
    }

    printf("\nCommands:\n");
    printf("  pwd\n");
    printf("  export NAME=value\n");
    printf("  child\n");
    printf("  exit\n");

    // --- Inferred Input Reading Logic (Cropped between images) ---
    printf("Enter command: ");
    if (fgetc(input, sizeof(input), stdin) == NULL) {
        break;
    }
    input[strcspn(input, "\n")] = '\0'; // Remove newline

    if (strcmp(input, "pwd") == 0)
    {
        printf("%s\n", cwd);
    }
    else if (strcmp(input, "exit") == 0)
    {
        printf("Exiting program...\n");
        break;
    }
    // -----

    /* Child Process */
    else if (strcmp(input, "child") == 0)
    {
        pid_t pid = fork();

        if (pid == 0)
        {
            execl("/usr/bin/env", "env", NULL);
            perror("exec");
            exit(1);
        }

        wait(NULL);
    }

    /* Export Command */
    else if (strcmp(input, "export ", 7) == 0)
    {
        char *expr = input + 7;
        char *eq = strchr(expr, '=');

        if (eq == NULL)
        {
            printf("Invalid syntax. Use: export NAME=value\n");
            continue;
        }

        *eq = '\0';
        char *name = expr;
        char *value = eq + 1;

        if (!isValidName(name))
        {
            printf("Invalid variable name.\n");
            continue;
        }

        setenv(name, value, 1);
        printf("Exported: %s=%s\n", name, getenv(name));
    }
}

return 0;
```

Output:

```
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ nano Skill10.c
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ gcc Skill10.c -o Skill10
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ ./Skill10

Current Directory: /home/chinn/OS_SKILL

Commands:
  pwd
  export NAME=value
  child
  exit
Enter command: pwd
/home/chinn/OS_SKILL

Current Directory: /home/chinn/OS_SKILL

Commands:
  pwd
  export NAME=value
  child
  exit
Enter command: export

Current Directory: /home/chinn/OS_SKILL

Commands:
  pwd
  export NAME=value
  child
  exit
Enter command: exit
Exiting program...
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ |
```

11.

```
chinn@LAPTOP-RDULKNPB: ~ × + ∨
GNU nano 8.7.1 Skill11.c *
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_CMD 100

int main() {
    char history[MAX_HISTORY][MAX_CMD];
    char command[MAX_CMD];
    int count = 0;

    printf("Simple Command History\n");
    printf("Type 'history' to display commands\n");
    printf("Type 'exit' to quit\n\n");

    while (1) {
        printf("shell> ");
        fgets(command, MAX_CMD, stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0) {
            printf("\nCommand History:\n");

            for (int i = 0; i < count; i++) {
                printf("%d %s\n", i + 1, history[i]);
            }

            continue;
        }

        if (strlen(command) == 0)
            continue;

        if (count < MAX_HISTORY) {
            strcpy(history[count], command);
            count++;
        } else {
            for (int i = 0; i < MAX_HISTORY - 1; i++) {
                strcpy(history[i], history[i + 1]);
            }

            strcpy(history[MAX_HISTORY - 1], command);
        }
    }

    return 0;
}
```

Output:

```
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ nano Skill11.c
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ gcc Skill11.c -o Skill11
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ ./Skill11
Simple Command History
Type 'history' to display commands
Type 'exit' to quit

shell> ls
shell> pwd
shell> date
shell> history

Command History:
1  ls
2  pwd
3  date
shell> exit
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ |
```

12.

```
chinn@LAPTOP-RDULKNPB: ~ × + ▾
GNU nano 8.7.1 Skill12.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd[2];
    pid_t p1, p2;

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    p1 = fork();

    if (p1 == 0) {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);

        execlp("ls", "ls", NULL);

        perror("execlp");
        exit(1);
    }

    p2 = fork();

    if (p2 == 0) {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);

        execlp("wc", "wc", "-l", NULL);

        perror("execlp");
        exit(1);
    }

    close(fd[0]);
    close(fd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    return 0;
}
```

Output:

```
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ nano Skill12.c
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ gcc Skill12.c -o Skill12
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ ./Skill12
34
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ |
```

13.

```
chinn@LAPTOP-RDULKNPB: ~ × + v
GNU nano 8.7.1 Skill13.c *
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        int fd = open("input.txt", O_RDONLY);

        if (fd < 0) {
            perror("open");
            exit(1);
        }

        dup2(fd, STDIN_FILENO);
        close(fd);

        execlp("wc", "wc", "-l", NULL);

        perror("execlp");
        exit(1);
    }

    wait(NULL);

    return 0;
}
```

Output:

```
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ nano Skill13.c
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ gcc Skill13.c -o Skill13
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ ./Skill13
open: No such file or directory
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$
```

14.

```
chinn@LAPTOP-RDULKNPB: ~ × + ∨
GNU nano 8.7.1 Skill14.c *
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        int out = open("output.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

        int err = open("error.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

        if (out < 0 || err < 0) {
            perror("open");
            exit(1);
        }

        dup2(out, STDOUT_FILENO);
        dup2(err, STDERR_FILENO);

        close(out);
        close(err);

        execlp("ls", "ls", "nonexistent_file", NULL);

        perror("ls");
        exit(1);
    }

    wait(NULL);

    printf("Output written to output.txt\n");
    printf("Error written to error.txt\n");

    return 0;
}
```

Output:

```
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ nano Skill14.c
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ gcc Skill14.c -o Skill14
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ ./Skill14
Output written to output.txt
Error written to error.txt
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ |
```

15.

```
chinn@LAPTOP-RDULKNPB: ~ × + ∨
GNU nano 8.7.1 Skill18.c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pipefd[2];

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 == 0) {
        int fd = open("result.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

        if (fd < 0) {
            perror("open");
            exit(1);
        }

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        execlp("ls", "ls", NULL);

        perror("ls");
        exit(1);
    }

    pid_t p2 = fork();

    if (p2 == 0) {
        int fd = open("result.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

        if (fd < 0) {
            perror("open");
            exit(1);
        }

        dup2(pipefd[0], STDIN_FILENO);
        dup2(fd, STDOUT_FILENO);

        close(fd);
        close(pipefd[0]);
        close(pipefd[1]);

        execlp("wc", "wc", "-l", NULL);

        perror("wc");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    printf("Result stored in result.txt\n");

    return 0;
}
```

Output:

```
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ nano Skill15.c
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ gcc Skill15.c -o Skill15
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ ./Skill15
Result stored in result.txt
chinn@LAPTOP-RDULKNPB:~/OS_SKILL$ |
```